JAVA · KOTLIN / SPRING 백엔드 포트폴리오

신지섭

wltjq1203@icloud.com
github.com/tlswltjq
2026.11

결제·환불처럼 실패와 재처리가 얽힌 흐름에서 데이터 정합성을 지키는 백엔드 개발자입니다.
장애를 직접 재현해 원인을 좁히고, 수정 효과를 같은 조건의 실험과 통합 테스트로 확인합니다.

8 → 0건
장애 재현 실험의 오환불 · ESD

138 → 480.5
Outbox 발행 처리량 events/s · ESD

1.18 → 0.56초
결제 후 이용 권한 확인 p95 · ESD

0 → 3/3건
동시 구매 성공 · 커넥션 고갈 해소 · SNG

프로젝트	해결한 문제	결과	쪽
ESD 개인	DB 장애 때 재시도 소진을 지급 실패로 판단해 정상 결제를 자동 환불	**오환불 8 → 0건**	3
	재시도 중인 이벤트보다 같은 주문의 후행 이벤트가 먼저 발행됨	순서 회귀 테스트 6종	4
	Outbox 발행에서 추적 정보가 끊겨 지연 구간이 보이지 않음	**p95 1.18 → 0.56초**	5
	적체 중에도 배치마다 폴링을 기다리고 배치 전체를 직렬 발행	**138 → 480.5 events/s**	6
SNG 개인	동시 구매 요청이 모두 추가 DB 커넥션을 기다림 (Kotlin)	**성공 0 → 3/3건**	8
	정산이 실패하면 진행 상태까지 롤백돼 재개 지점이 없음	**7,000건 보존**	9
JobMate 팀 5명	영상 업로드 완료 전 글 제출 시 404 · 변환 결과물 재입력	레코드 선생성 · 경로 분리	11

언어
Java 21 · Kotlin

백엔드
Spring Boot 3 · JPA/Hibernate · Spring Security · WebSocket/STOMP

데이터·메시징
MySQL 8 · Redis · Apache Kafka · AWS S3

테스트·운영
JUnit 5 · Testcontainers · k6 · Docker Compose · Grafana Tempo

ESD 플랫폼 · 개인 프로젝트 · 2026.08–

게임 등록부터 구매·다운로드·환불·정산까지 이벤트로 연결

스튜디오·스토어·주문·결제·라이선스·다운로드·정산 등 9개 서비스를 서비스별 DB와 Kafka 이벤트로 연결했습니다. 결제·지급·정산을 한 트랜잭션으로 끝낼 수 없어, 업무 데이터와 이벤트를 **같은 트랜잭션에 저장**한 뒤 Outbox로 보내고 소비 측에서 중복을 확인합니다. 아래 사례는 **결제는 성공했지만 지급은 실패한 상황**에서 출발합니다.



구간	서비스	주요 책임
등록·출시	studio · review · catalog	파일 검증, 심사 이력, 공개 빌드·상품 확정
조회·구매	store · order · payment	조회 모델, 서버 가격 계산, 결제·환불
이용·정산	license · download · settlement	라이선스 지급·회수, 다운로드 권한, 원장

API 데모 · 빈 환경에서 두 회차
가입부터 등록·심사·출시·구매·환불·월 마감까지 실행했습니다. 내려받은 ZIP과 업로드 원본의 SHA-256 일치, 환불 후 다운로드 권한 회수, 월 정산 대사 일치를 확인했습니다. 회차별 검증 57개·상태 검사 16개 통과.

설계 판단 · AI가 쟁점과 대안을 내면 제가 고르고, 구현·테스트는 AI가 맡았습니다

쟁점	AI 추천	제 판단과 근거	결과 · 감수한 비용
지급보다 먼저 도착한 환불 정지 요청	License가 정지 의도를 보관	서로 신뢰하지 않는 별개 서비스인데 License가 외부 환불 요청을 계속 보관하게 된다고 반박. 근거를 가진 Order가 기다리는 안을 제안	Order가 지급 확인 뒤 정지 요청 감수: 지급부터 정지까지의 짧은 접근 틈
주문 만료(제가 제안) 직전에 시작한 승인이 기한 뒤 끝남	기한 뒤 승인은 자동 취소	주문이 만료되지 않았다면 외부 승인까지의 지연은 서비스가 부담해야 한다고 반문(환불 기한 판정과 같은 원칙)	30분은 서버가 승인 진행을 확정하는 기한 감수: 승인 진행 기록과 만료 경합 처리

Java 21 · Spring Boot 3.5 · MySQL · Kafka · Redis · Elasticsearch · MongoDB · Docker · Grafana Tempo · 서비스 구성 · 설계 결정 기록 · API 데모 절차·기록 (2026.09.30, 7873593) · 외부 PG·상점 화면은 데모에서 제외
2026.10.15–11.06 대화 원본에서 확인한 사용자 결정 123건 중 직접 제안 14 · AI 주장 정정 8 · 추천과 다른 선택 6건, 나머지 대부분은 추천 수용 · 판단 색인(원문 인용) · 결정 29 · 결정 32

ESD · 사례 1 · 장애 시 환불 판단

같은 장애 조건에서 오환불을 8건에서 0건으로 줄임

문제 라이선스 테이블 접근만 60초 차단하자 **정상 결제 8건이 자동 환불**됐습니다. (주문 40건 · 초당 2건)

원인

스키마 전체를 차단하면 Outbox 저장도 실패해 문제가 가려졌습니다. 테이블만 막으면 보상 이벤트는 저장됐고, **재시도 소진을 지급 실패로 판단**한 결과가 환불로 이어졌습니다.

해결

환불 전에 **실패 원인과 실제 지급 이력**을 확인합니다. 저장소 장애, 이미 지급된 주문, 지급 이력 조회 실패는 보류(DLT)하고, 확인 가능한 지급 실패만 보상 경로로 보냅니다.

선택 근거 · 한계

블로킹 재시도를 길게 유지하면 같은 파티션의 후속 주문도 기다리므로, 보류 후 재처리를 택했습니다. 자동 재투입은 같은 오류를 반복할 수 있어 원인 해결 후 운영 API로 넣습니다. 대신 보류 건 확인·재처리가 운영 부담으로 남습니다.

검증

같은 장애 조건	수정 전	수정 후
정상 지급	32건	33건
보류	0건	7건
오환불	8건	**0건**

보류 건 재투입

장애 해소 후 보류된 7건을 운영 API로 재투입했고, **1초 이내에 7건 모두 지급**됐습니다. 환불·보류 조건은 테스트로 고정했습니다.

2026.08 검증 · D-027·D-028 재현 기록 · 장애 주입 조건·결과 · KafkaErrorHandlerConfig · 환불·보류 조건 테스트

ESD · 사례 2 · 이벤트 발행 순서

실패한 이벤트를 건너뛰지 않고 같은 주문 안에서 순서를 지킴

문제 **결제 완료 이벤트 발행이 실패한 뒤 같은 주문의 결제 취소가 먼저 발행돼, 늦게 온 완료 이벤트로 라이선스가 지급될 수 있었습니다.**

원인

발행기가 실패한 이벤트를 건너뛰고 다음 이벤트를 계속 보냈습니다. 메시지 키를 주문번호로 지정해도 **Kafka에 보내기 전에 바뀐 순서**는 복구되지 않습니다. 또 선행 이벤트만 재시도 시각을 늦추면, 다음 조회에서 후행 이벤트만 선택될 수 있었습니다.

해결

주문번호로 이벤트를 묶고, 같은 주문은 앞선 발행의 응답을 확인한 뒤 다음을 보냅니다. 실패하면 **후행 이벤트에도 선행 이벤트의 재시도 시각을 적용**해 함께 보류하고, 다른 주문은 병렬로 계속 발행합니다.

선택 근거 · 한계

같은 주문을 릴레이 한 대가 처리하는 조건에서의 보장입니다. 다중 릴레이로 늘리면 주문별 담당 분리가 추가로 필요합니다. DEAD로 넘어간 이벤트의 재투입 순서는 별도 복구 절차에서 다룹니다.

검증 발행 순서 테스트와 백오프 조회 테스트는 각각 통과했지만 두 조건이 만나는 동작은 확인하지 못했습니다. 테스트 대역에 실제 조회 시간 조건을 넣자 **다음 배치에서 후행 이벤트가 앞서는 문제가 재현**됐고, 아래 6개 상황을 회귀 테스트로 고정했습니다.

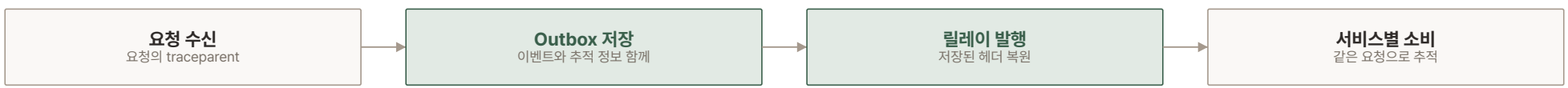
동일 주문 정상 발행 → 적재 순서대로	선행 발행 실패 → 후행 이벤트 보류	다른 주문 → 실패와 별도로 계속 발행
재시도 시각 전 재조회 → 후행이 앞서지 않음	순서 때문에 보류 → 재시도 횟수 미차감	선행이 DEAD 전환 → 다음 회차에 후행 진행

D-014 · 테스트가 놓친 조회 조건 · OutboxRelayTest · publishPreservingOrder / holdRemainder · 보장 범위·대안 검토

ESD · 사례 3 · 이벤트 추적과 지연

끊긴 추적을 이어 지연 구간을 찾고, 발행 대기를 1초에서 200ms로 줄임

문제 결제 승인 요청부터 이용 권한 확인까지 **p95 1.30초·1.06초**였지만, 서비스 사이 어디서 기다리는지 보이지 않았습니다.



원인

Outbox는 요청이 끝난 뒤 별도 스케줄러가 발행하므로 **요청의 추적 정보가 전달되지 않았습니다**. 자동 계측 후에도 주문 요청 대신 릴레이 회차의 traceId가 남았습니다. 추적을 연결하자 order·license·settlement까지 +527ms, license 이후 서비스까지 +759ms의 대기가 보였습니다.

해결

Outbox 행에 이벤트와 함께 `traceparent`를 저장하고, 릴레이가 이를 Kafka 헤더로 복원해 **6개 서비스를 같은 요청으로 연결**했습니다. 이어 확인한 발행 대기를 줄이려 Outbox 폴링 주기를 **1초 → 200ms**로 바꿨습니다.

선택 근거 · 한계

이벤트와 추적 정보가 함께 커밋되고, 이벤트 본문에 추적 도구의 필드를 넣지 않아도 됩니다. 주기를 줄이면서 MySQL CPU 평균이 7.3% → 8.8%로 늘어 이를 감수하고 200ms를 기본값으로 했습니다. 권한을 0.5초마다 조회해 더 짧은 지연과 큰 부하의 한계는 이 측정으로 구분하지 못합니다.

검증

폴링 주기	1회 p95	2회 p95	평균
1000ms	1.30초	1.06초	1.18초
200ms	563ms	561ms	**0.56초**

폴링 주기 비교 측정 원본 · 2026.08.10 · 측정 시작·종료 코드 · 추적 헤더 복원 코드 · 기준 변경 `7574ea2`

k6 5 VU · 2분 · 조건별 2회. 완료 건수는 회차별 252·198건에서 285·286건으로 늘었고, 네 회차 모두 성공률 100%였습니다. 헤더 복원은 발행 성공·재시도와 추적 정보가 없는 이벤트를 테스트했습니다.

ESD · 사례 4 · Outbox 발행 처리량

직렬 범위를 주문 단위로 좁히고 배치를 연속 처리해 처리량 3.5배

문제 이벤트가 쌓여 있어도 발행기가 **배치마다 다음 폴링(1초)을 기다렸고**, 배치 전체를 직렬로 발행했습니다.

원인

순서를 지키려고 배치 전체를 하나의 직렬 범위로 다뤘습니다. 서로 다른 주문의 이벤트도 앞선 발행을 기다렸습니다. 또 한 번의 실행에 배치 하나만 처리해, 적체 중에도 배치 사이마다 폴링 대기가 생겼습니다.

해결

직렬 범위를 **주문번호 단위**로 좁혀, 같은 주문은 사례 2처럼 순서대로, 다른 주문은 병렬로 발행했습니다. 한 번의 실행에서 배치별 트랜잭션으로 **최대 10개 배치를 연속 처리**해 배치 사이의 대기를 줄였습니다.

선택 근거 · 한계

폴링 주기는 그대로 두고, 쌓인 이벤트가 있을 때만 연속으로 처리합니다. 개선 기여는 키별 병렬화 약 1.13배, 배치 연속 처리 약 3.08배로 나눠 확인했습니다. 발행기 1개 기준의 측정이며 HTTP 처리량이나 현재 시스템 성능을 뜻하지 않습니다.

검증

초기 실험 · 두 변경을 단계별로 측정

단계	events/s
기존	138
주문 단위 병렬 발행	156.1
+ 배치 연속 처리	**480.5**

2026.08 · macOS 8GB · Docker 4GB · 서로 다른 키 2만 건 적재

재측정 · 배치 연속 처리만 비교

연속 배치 수	1회	2회	평균
1개	180.7	180.7	180.7
10개	975.6	921.7	**948.7**

OCI Ampere A1 4코어 · 2만 건 · 배치 200건 · 모든 회차 실패·DEAD 0건

처리량·지연 측정 표 · ESD PR #11(Outbox 개선), #66(성능 기여 구분) · OutboxRelay

SNG · 개인 프로젝트 · 2026.06–07 · Kotlin

게임 서버와 웹 상점의 구매·보상 처리 구현

Kotlin/Spring 기반 게임 서버와 웹 상점입니다. 웹 상점의 마일리지 구매·보상 전달과, 게임 서버의 시즌 순위 저장·보상 우편 지급을 구현했습니다. 두 사례 모두 **테스트는 통과했지만 실제 트랜잭션에서 다르게 동작한 문제**입니다.

구간	처리하는 업무	검증한 문제	쪽
웹 상점	구매 한도 확인 · 주문 저장 · 마일리지 차감	동시 구매의 커넥션 대기 · 잔액과 원장 정합성	8쪽
게임 서버	시즌 순위 저장 · 보상 우편 지급 · 정산 재개	실패 후 진행 상태 보존 · 재개 시 중복 지급	9쪽

동시 구매가 멈춘 원인
주문 트랜잭션 안에서 초기화용 트랜잭션을 따로 열어 커넥션을 추가로 요구했습니다. 동시 요청이 풀을 모두 점유하자 전부 다음 커넥션을 기다렸습니다.

재개 테스트가 놓친 롤백
목 테스트는 재개 상태를 직접 만들어 통과했습니다. 실제 실패에서는 그 상태까지 롤백돼 이어갈 지점이 남지 않았습니다.

Kotlin · Spring Boot · JPA · MySQL · Redis Streams · Docker · JUnit 5 · Testcontainers
비공개 저장소 · 기준 `1292702` · 권한이 없어도 확인할 수 있도록 핵심 코드와 검증 조건을 다음 두 쪽에 수록했습니다.

SNG · 사례 1 · 동시 구매

초기화를 주문 트랜잭션에 통합해 커넥션 고갈을 없앰

문제 CI의 마일리지 동시 구매 테스트가 실패했습니다. 재현해 보니 **요청 10건이 모두 커넥션 획득 중 대기**했고 성공은 0건이었습니다.

원인

주문 트랜잭션이 커넥션을 쥔 채 계정 초기화를 `REQUIRES_NEW`로 호출해 커넥션을 하나 더 요구했습니다. 동시 주문 10건이 풀 10개를 점유하자, 반환할 주문도 진행하지 못했습니다.

```
@Transactional fun place(...) { ensureAccount(...) }
@Transactional(propagation = REQUIRES_NEW)
fun ensureAccount(...) { ... }
```

해결

초기화를 주문 트랜잭션에 통합하고, 중복 생성은 `INSERT IGNORE`로 처리해 추가 커넥션을 없앴습니다. 기존 행에도 삽입하면 공유 잠금이 차감용 잠금으로 바뀌며 경합해, **이미 있는 행은 삽입을 건너뛰도록** 보완했습니다.

```
if (repo.existsById(id)) return
repo.insertIfAbsent(id, now())
```

선택 근거 · 한계

초기화를 분리했던 이유는 동시 생성 시 중복 키 예외가 주문 전체를 롤백시키지 않게 하려는 것이었습니다. 중복을 예외 없이 처리하면 분리할 필요가 없습니다. 배치 적립처럼 실패 격리가 필요한 경로의 `REQUIRES_NEW`는 유지했습니다.

검증

잔액 300 · 상품 100 · 동시 10건	수정 후
구매 성공 (기대 3건)	**3건**
최종 잔액	**0**
잔액과 원장 합계	**일치**

Testcontainers의 MySQL 8.0과 Flyway 스키마를 쓰는 통합 테스트입니다. 수정 전 CI에서 실패한 테스트가 수정 후 통과했습니다.

ConcurrencyIntegrationTest.mileageSpendCannotOverdraw · `338ef8f` 실패 → `295ee701` 통과 · PlaceOrder / MileageWriter / PurchaseLimitGuard · 코드는 요약 발췌

SNG · 사례 2 · 정산 재개

재개 테스트가 놓친 상태 롤백을 트랜잭션 경계로 해결

문제 정산 중 실패하면 **다시 시작할 지점이 남지 않았습니다.** 그런데 재개 테스트는 통과하고 있었습니다.

원인

정산 전체를 한 트랜잭션으로 감쌌습니다. 목 테스트는 `SETTLING` 상태를 직접 만들어 재개를 확인했지만, 실제 실패에서는 **그 상태와 지급 기록도 함께 롤백**됐습니다.

해결

전체 트랜잭션을 없애고 단계별로 커밋했습니다. 200건씩 보상 우편과 `SENT` 상태를 같은 트랜잭션에 저장하고, 완료 행은 조회에서 빠지므로 항상 첫 페이지를 조회합니다. 재개 시에는 남은 `PENDING`만 처리합니다.

선택 근거 · 한계

실패한 청크는 우편과 상태가 함께 롤백돼 지급과 상태가 어긋나지 않습니다. 목 테스트를 실제 트랜잭션을 쓰는 테스트로 바꾸고, 비교용 단일 트랜잭션 구성을 같은 조건에 두었습니다. H2 예외 주입 검증이며, 프로세스 강제 종료·MySQL 잠금 경합은 별도로 검증해야 합니다.

검증

1만 건 · 7,001번째에 예외	단일 트랜잭션	청크 커밋
중단 후 지급·우편	0건	**7,000건**
중단 후 시즌 상태	ACTIVE	**SETTLING**
재개 후 추가 지급	—	**3,000건**
재개 후 총 우편	—	**10,000건**

보상 계산 7,001번째에 예외를 주입해 두 구성을 비교했습니다. 청크 커밋은 진행을 보존하고, 재개 시 중복 지급 없이 남은 3,000건만 지급했습니다.

`b44f7e4` 단일 트랜잭션 → `e83f23b` 단계별 커밋 · 비교 테스트 `23dda60` · SettleSeasonBeforeAfterTest · 기준 `1292702`

JobMate · 팀 5명 · 2025.09–10 · 영상·P2P 담당

S3 직접 업로드 · 별도 워커 영상 변환 · 상태 조회 구현

프로그래머스 백엔드 데브코스의 IT 진로 탐색·멘토링 서비스입니다. 영상 API·변환 워커와 P2P 시그널링을 담당했고, 프론트엔드 개발자에게 API 구현 방식을 공유하며 업로드·상태 조회 기능을 연동했습니다.



담당 구현	처리하는 작업	다음 단계로 전달하는 값
Spring API	권한·확장자 확인, 영상 레코드 관리, 업로드·재생 URL 발급	UUID · 업로드 URL · 화질별 상태
HTTP 브리지	SNS 구독 확인, S3 객체 생성 메시지 파싱, Kafka 발행	bucket · object key → s3-events
Python 워커	ffprobe 검사, 1080p·720p·480p 변환, 결과 업로드	transcoding-status에 화질별 상태 발행

직접 업로드와 별도 워커를 둔 이유

파일 전송과 CPU를 쓰는 변환을 API 요청에서 분리했습니다. 워커가 대기·진행·완료·실패 상태를 Kafka로 보내면 Spring이 UUID에 연결해 저장하고, 클라이언트는 완료 여부를 조회해 재생 URL을 요청합니다.

P2P 시그널링 인증과 동료 지원

HttpOnly 쿠키의 토큰을 핸드셰이크 세션에 보관하고 STOMP 연결 시 검증해 발신자를 `Principal`로 식별했습니다. 테스트 HTML·시연 화면과 변경된 API 경로, 로컬 테스트 조건을 PR #184에 공유했습니다.

Java 21 · Spring Boot · Spring Security · WebSocket/STOMP · AWS S3/SNS · Kafka · Python · ffmpeg · Docker

영상 API · HTTP 브리지 · 변환 워커 · 컨테이너 구성

업로드 완료 전 글 제출 404를 고치고, 변환 결과물의 재입력을 막음

업로드 완료 전 글 제출 시 404

문제 영상 업로드가 끝나기 전에 뉴스를 제출하면 404가 발생했습니다.

원인 워커의 상태 메시지를 받은 뒤에야 영상 레코드를 만들어, 그 전에는 참조할 레코드가 없었습니다.

해결 업로드 URL 발급 시 `READY_TO_UPLOAD` 상태로 레코드를 먼저 만들고, 이후 상태 메시지로 갱신합니다. 뉴스는 완료 전에도 UUID로 영상을 참조합니다.

한계 URL 발급 테스트는 FileManager를 모킹해, 이 오류의 회귀 검증은 포함하지 않습니다.

변환 결과물의 재입력 방지

문제 원본과 결과물을 같은 버킷에 저장하도록 바꾸면서, 결과물이 다시 변환될 수 있었습니다.

원인 경로를 구분하지 않으면 결과물의 S3 객체 생성 알림도 원본과 같은 이벤트로 워커에 전달됩니다.

해결 원본은 `videos/`, 결과물은 `transcoded/` 경로로 나누고 결과물 이벤트는 다운로드 전에 제외했습니다.

한계 같은 원본 이벤트가 중복 전달될 때의 멱등 처리는 추가로 필요합니다.

```
if key.startswith("transcoded/"):
    continue  # 변환 결과물은 처리하지 않음
```

업로드 완료 전 제출 실패 이슈 · 레코드 생성 시점 수정 · 결과물 재입력 제외 · 영상 API 테스트 · 워커를 포함한 종단 검증은 별도로 필요합니다.

근거 · 다음 작업 · 연락처

사례별 코드 · 테스트 · 실험 기록

사례	근거 자료	기준	쪽
ESD 설계 판단	판단 색인(사용자 발언 원문) · 결정 29 · 결정 32 · 검증 근거 색인	`265940e`	2쪽
ESD 구조 검사	아키텍처 검사 점검·정비 기록 (구조 검사 615건) · 결정 44	`0cc225b`	—
ESD 오환불	D-027·D-028 원인·수정 · 장애 실험 · 환불·보류 조건 테스트	`a525243`	3쪽
ESD 발행 순서	보장 범위와 대안 · OutboxRelayTest	`a525243`	4쪽
ESD 추적·처리량	traceparent 복원·연속 배치 · 측정 표 · PR #11 · #66	원본 `17786a5`	5쪽
SNG 동시 구매	MileageWriter · PurchaseLimitGuard · ConcurrencyIntegrationTest (MySQL)	`338ef8f` → `295ee701`	8쪽
SNG 정산	단일 트랜잭션 제거 · SettleSeasonBeforeAfterTest (H2 예외 주입 비교)	`e83f23b` · `1292702`	9쪽
JobMate 영상	URL·상태 관리 · 변환 워커 · 영상 테스트 · 상태 연동 · 브리지 수정	`a4e2d35`	11쪽

현재 진행 · ESD MVP 재작성

10월에는 모듈별 책임·계약·완료 조건을 정리해 11개 모듈의 MVP를 재작성했습니다. 모의 PG를 포함한 실제 프로세스 검증 32단계를 통과했고, OCI에서 방식별 3회 비교해 이미지 생성·게시 시간 중앙값을 65.4% 줄였습니다. 이 문서의 장애·성능 실험은 8–9월 구현의 결과입니다.

문서 기준 2026.11 · 현재 MVP 검증 · 이미지 생성·게시 비교 · 첫 쪽으로

연락처

신지섭 · Java·Kotlin / Spring 백엔드
wltjq1203@icloud.com
(+82) 10-6619-9035
github.com/tlswltjq